Increasing Reward Quality & Density for Verifiable Rewards

Ana Cunha, Leonard Evers, Leonhard Driesch

Technical University of Munich

June 24th, 2026



Outline

1 Introduction

2 Related Work

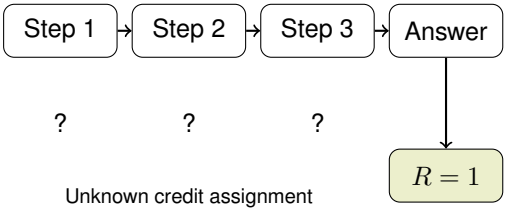
3 Progress

4 Outlook

5 References

Motivation

- RL for LLMs often suffers from **reward sparsity**.
- Particularly severe for **binary verifiable rewards**.
- The model receives only a final **0/1 reward**.
- Intermediate reasoning steps receive no direct feedback.



Key Challenge

How can we identify which intermediate reasoning steps deserve credit when only the final outcome is observed?

Our Approach

Goal

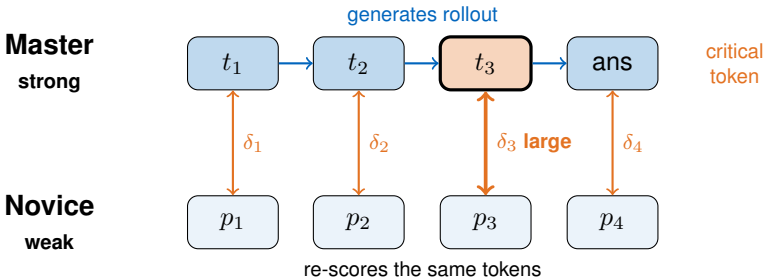
Increase reward density for binary verifiable tasks by identifying **critical reasoning steps** within successful trajectories.

Key Idea: Master/Novice Credit Assignment

- Start from a strong LLM (the **Master**) and construct a weaker variant (the **Novice**).
- Compare the action probabilities predicted by both models at every reasoning step.
- Large divergences indicate steps where the Master uses knowledge or reasoning unavailable to the Novice.
- These steps receive higher reward, while agreement corresponds to routine or uninformative transitions.

Our Approach

Master/Novice divergence as a reward signal



Reward signal

Large divergence δ_t $\Rightarrow$ the token relies on knowledge the Novice lacks $\Rightarrow$ higher credit.

Outline

1 Introduction

2 Related Work

3 Progress

4 Outlook

5 References

Turning Partial Success into Dense Verifiable Rewards

Core problem in code RL

- Verifiable rewards enforce **correctness via tests**
- But they are **extremely sparse** (only pass/fail)

Existing alternatives

- External reward models $\rightarrow$ dense but **misaligned + expensive**

Key observation

Partial test success already exists $\rightarrow$ unused signal

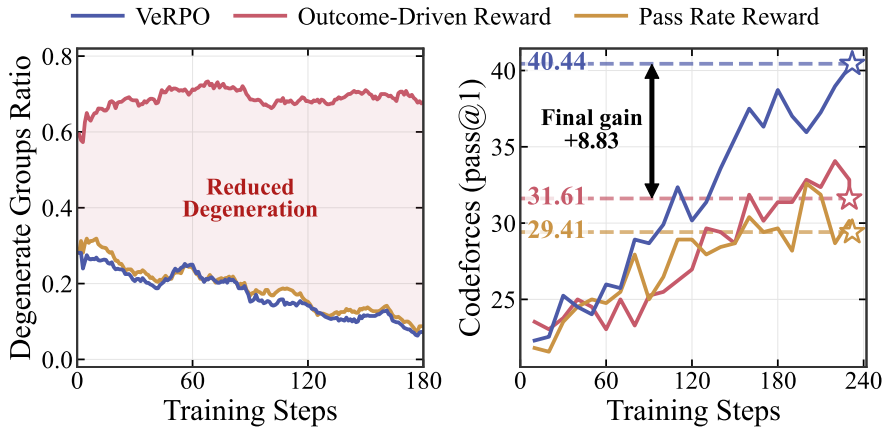
Idea

Can we use pass rate as a dense reward signal?

Instead of binary success, use fraction of passing tests as supervision?

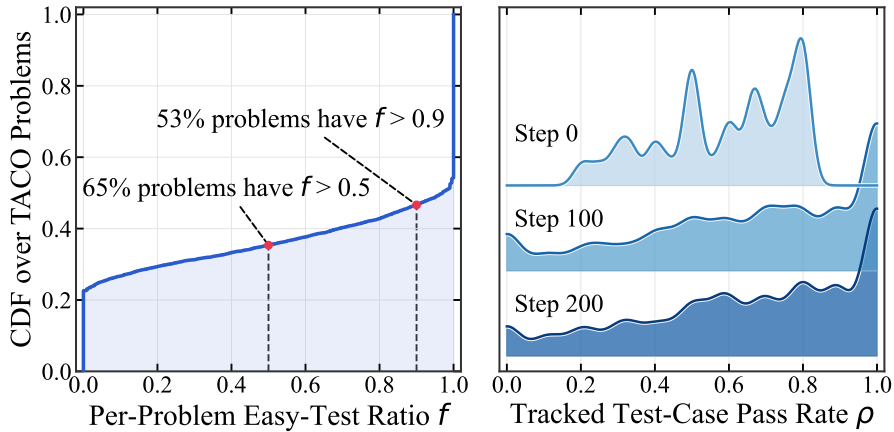
Turning Partial Success into Dense Verifiable Rewards

Answer:



Turning Partial Success into Dense Verifiable Rewards

Problem:



VeRPO: Turning Partial Success into Dense Rewards

Idea: combine two complementary signals

1. Local (dense signal)

- Use **test-case partial successes**
- Reward each step based on pass rate
- Captures fine-grained progress

2. Global (verifiable outcome)

- Final **pass/fail outcome**
- Down-weight long trajectories
- Ensures correctness signal

$$R^{turn}(\tau_i, t) = \sum_{j=1}^{|U_x|} w'_j \cdot p_{t,i}^{(j)}$$

$$\tilde{R}^{traj} = R^{traj}(\tau_i) \cdot \gamma^{|\tau_i|}$$

VeRPO: Constructing Learning Signals

Local (turn-level) signal

- Uses partial test-case success
- Provides step-level supervision

Global (trajectory-level) signal

- Uses decayed outcome reward
- Captures overall correctness

Local advantage

$$A^{\text{turn}}(\tau_i, t) = R^{\text{turn}}(\tau_i, t) - \text{mean}(G(\mathcal{G}_x))$$

Global advantage

$$A^{\text{traj}}(\tau_i) = \tilde{R}^{\text{traj}}(\tau_i) - \text{mean}(\{\tilde{R}^{\text{traj}}(\tau_j)\}_{j=1}^{N})$$

VeRPO: Turn-Level Training Objective

Final step: combine global correctness with local progress

Turn-level advantage

The trajectory-level signal is broadcast to each step and combined with the local advantage:

$$A(\tau_i, t) = A^{traj}(\tau_i) + \beta \cdot A^{turn}(\tau_i, t) \tag{1}$$

Interpretation

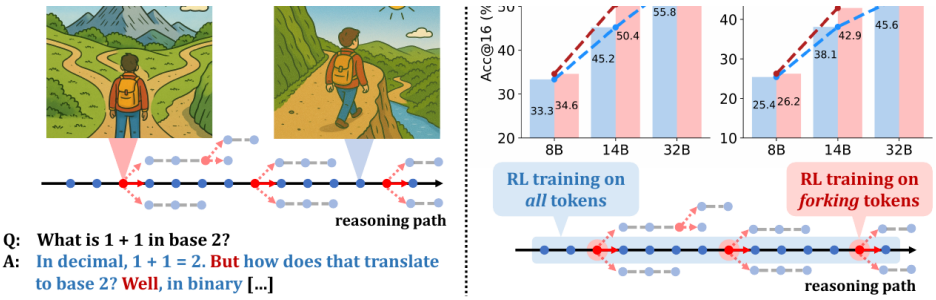
- A^{traj} ensures global correctness (did the program solve the task?)
- A^{turn} provides dense step-level credit assignment
- β controls the trade-off between exploration and precision

Turning Partial Success into Dense Verifiable Rewards

Method	Train	RM	Reward	HumanEval				BigCodeBench				LCB		Codeforces		Avg	
				-		Plus		Full		Hard		V6		CodeElo			
				ST	MT	ST	MT	ST	MT	ST	MT	ST	MT	ST	MT	ST	MT
Qwen3-8B	-	-	-	91.03	95.05	87.49	90.14	35.85	58.32	16.38	41.69	27.12	28.14	20.77	22.73	46.44	56.01
GRPO	ST	-	0-1	91.99	96.34	88.03	90.32	36.33	57.87	16.87	40.54	28.35	29.28	28.49	30.72	48.34	57.51
	MT	-	0-1	91.46	96.41	87.57	91.23	35.73	60.91	16.55	43.32	28.35	30.50	27.38	31.61	47.84	59.00
AceCoder	ST	7B	Dense	92.04	96.59	88.71	91.46	36.07	58.36	14.86	39.53	27.78	27.57	25.06	26.96	47.42	56.74
	MT	7B	Dense	---------- Training Collapse ----------													
VeRPO	ST	-	Dense	**92.73**	96.49	**89.32**	91.84	**37.42**	58.95	**17.91**	41.55	**29.72**	30.14	**30.50**	33.76	**49.60**	58.79
	MT	-	Dense	91.69	**97.87**	88.05	**93.14**	36.10	**62.52**	17.22	**45.69**	29.07	**33.07**	29.03	**40.44**	48.53	**62.12**

Beyond the 80/20 Rule

High-entropy “forking” tokens



- Only a **minority of CoT tokens are high-entropy** $\rightarrow$ they act as forks that choose the reasoning path.
- RLVR training mostly adjusts exactly these high-entropy tokens.

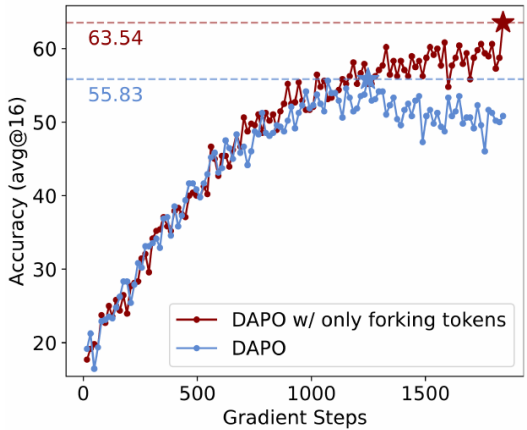
Beyond the 80/20 Rule

20% of tokens is enough

- Restrict policy-gradient updates to the top **20% highest-entropy** tokens.
- Matches full-gradient training on Qwen3-8B and **beats** it at scale (32B: **+11.0** AIME'25).
- Training on the low-entropy 80% instead hurts performance.

Takeaway

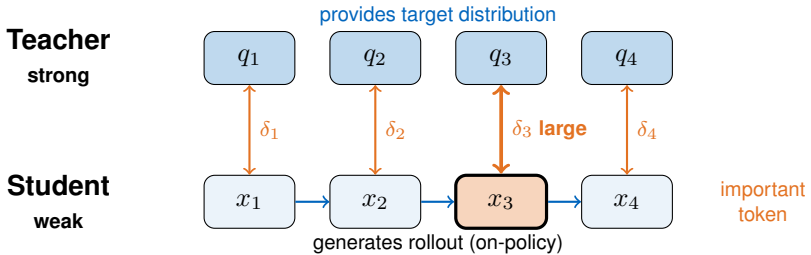
The learning signal is concentrated in a sparse set of decisive tokens.



(a) AIME'24 scores trained from Qwen3-32B base.

TIP: Token Importance in On-Policy Distillation

The idea

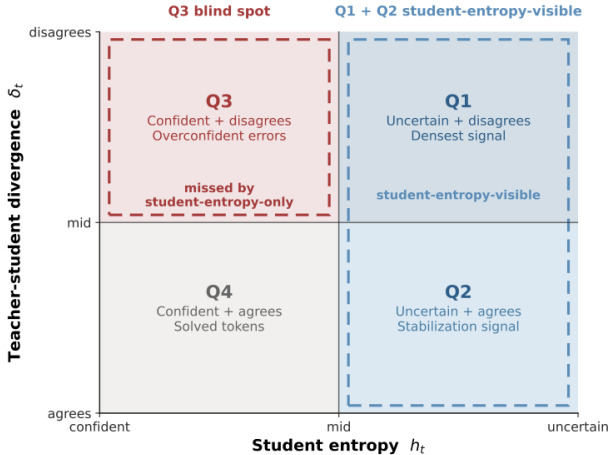


Token importance

Combine student uncertainty h_t and divergence δ_t $\Rightarrow$ distill the tokens carrying knowledge the student lacks.

TIP: Token Importance in On-Policy Distillation

Which tokens carry the signal?



TIP: Token Importance in On-Policy Distillation

Results:

Table 5: **Main results: Baseline vs. Entropy-only vs. Soft-OR.** Accuracy (%, mean@16 ± std). Soft-OR uses $s_t = \hat{h}_t + \hat{\delta}_t - \hat{h}_t \cdot \hat{\delta}_t$ (Eq. 5) with Top-K selection. Bold marks the best per benchmark.

		Baseline	Entropy-only		Soft-OR	
Model pair	Benchmark	100%	50%	20%	50%	20%
Qwen3-8B → 4B	MATH-500	76.7 ± 0.7	78.6 ± 0.6	74.1 ± 0.9	**79.1** ± 0.8	77.6 ± 0.7
	AIME'24	21.9 ± 1.2	23.8 ± 1.3	22.5 ± 1.1	**25.7** ± 1.4	24.5 ± 1.2
	AIME'25	19.4 ± 1.1	20.7 ± 1.3	21.5 ± 1.2	21.9 ± 1.2	**23.2** ± 1.2
Llama-70B → 8B	MATH-500	71.0 ± 0.7	74.0 ± 0.8	73.6 ± 0.7	**74.7** ± 1.0	74.2 ± 0.7
	AIME'24	21.5 ± 1.1	25.3 ± 1.5	18.8 ± 1.3	**26.0** ± 1.4	21.0 ± 1.5
	AIME'25	4.9 ± 0.9	7.5 ± 1.1	10.0 ± 1.2	**11.5** ± 1.1	10.9 ± 1.4
Qwen2.5-14B → 1.5B	MATH-500	55.1 ± 0.9	54.9 ± 0.9	54.0 ± 0.9	**56.2** ± 1.2	55.8 ± 0.9
	AIME'24	2.4 ± 0.7	3.3 ± 1.4	4.6 ± 1.3	3.8 ± 1.2	**5.0** ± 1.3
	AIME'25	**2.1** ± 0.9	1.0 ± 0.5	1.0 ± 0.6	1.5 ± 0.7	1.8 ± 0.6

PRIME: Process Reinforcement through Implicit Rewards

Motivation

Problem

- Outcome rewards are sparse: only the final answer is scored.
- Process rewards would give denser credit assignment.
- But standard PRMs need step labels, online updates, and extra training.

PRIME's goal

- Get dense token-level rewards.
- Use only outcome labels.
- Avoid a separate PRM training stage.

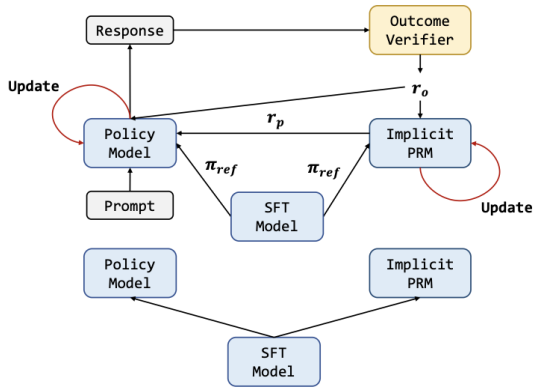


Figure 1. PRIME initializes the policy and implicit PRM from the same SFT model, then updates both online using outcome labels.

PRIME: Process Reinforcement through Implicit Rewards

Implicit Process Rewards

Core idea: train an ORM so it can also act as a PRM at inference time.

$$r_\phi(y_t) = \beta \log \frac{\pi_\phi(y_t \mid y_{<t})}{\pi_{\text{ref}}(y_t \mid y_{<t})}$$

- Reward is a **log-ratio** between a learned model and a reference.
- This gives a **dense token-level signal**.
- Training still uses only **binary outcome labels**.
- Online updates reduce distribution shift and reward hacking.

Takeaway: dense process supervision without manually labeled steps.

PRIME: Process Reinforcement through Implicit Rewards

Training Signal

PRIME combines two reward sources:

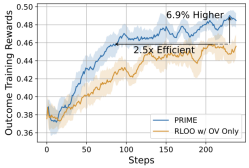
$$A_t = \underbrace{A_t^{\text{process}}}_{\text{dense token-level signal}} + \underbrace{A^{\text{outcome}}}_{\text{final correctness signal}}$$

- **Process term:** assigns credit across reasoning tokens.
- **Outcome term:** keeps optimization tied to final correctness.
- Both terms use RLOO-style normalization across rollouts.

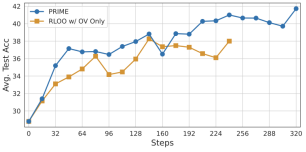
Why it matters: PRIME keeps the reliability of verifiable rewards while making them denser.

PRIME: Process Reinforcement through Implicit Rewards

Results



(a) Outcome training rewards (10-step moving).



(b) Test accuracy across different gradient steps.

Figure 3: **The effect of dense reward.** We compare PRIME and RLOO with outcome verifier (OV). PRIME leads to 2.5× sample efficiency (wall clock as X axis can be found in Figure 17) and 6.9% performance improvement. PRIME also substantially outperforms RLOO on downstream tasks.

Table 1: Detailed results of PRIME and RLOO w/ outcome verifier (OV). At the same 240 steps, the model trained by PRIME is generally better than the model trained by outcome rewards. We also reported avg@16 results in Table 12.

Method	Step	AIME 2024	AMC	MATH-500	MinervaMath	OlympiadBench	LeetCode	LiveCodeBench	Avg.
GPT-4o	-	9.3	45.8	76.4	36.8	43.3	58.9	48.8	45.6
Llama-3.1-70B-Inst.	-	20.0	37.3	65.0	37.1	30.5	35.0	34.4	37.0
Qwen2.5-Math-7B-Inst.	-	13.3	50.6	79.8	34.6	40.7	11.7	11.3	34.6
Eurus-2-7B-SFT	0	3.3	30.1	66.2	32.7	29.8	21.7	17.8	28.8
RLOO w/ OV Only	240	**20.0**	47.0	73.2	36.4	35.4	28.3	26.7	36.9
Eurus-2-7B-PRIME	80	20.0	41.0	68.2	38.2	37.0	26.7	26.6	36.8
	160	13.3	42.2	72.0	37.1	38.7	26.7	25.6	36.5
	240	**20.0**	**50.6**	**78.2**	**39.3**	**40.3**	**31.1**	**27.5**	**41.0**
	320	16.7	51.8	77.8	39.7	41.5	36.1	28.5	41.7
	592	26.7	57.8	79.2	38.6	42.1	33.3	28.6	43.9

Key result: PRIME is more sample-efficient than outcome-only RL and improves the SFT baseline substantially.

Outline

1 Introduction

2 Related Work

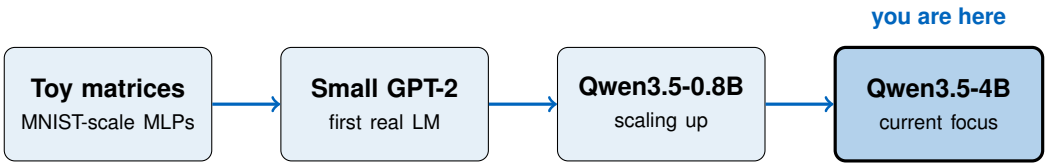
3 Progress

4 Outlook

5 References

Progress

From toy matrices to a 4B model



- Built on existing tensor-disentanglement code; first validated the effect on tiny weight matrices.
- Scaled progressively to confirm the approach holds on real language models.

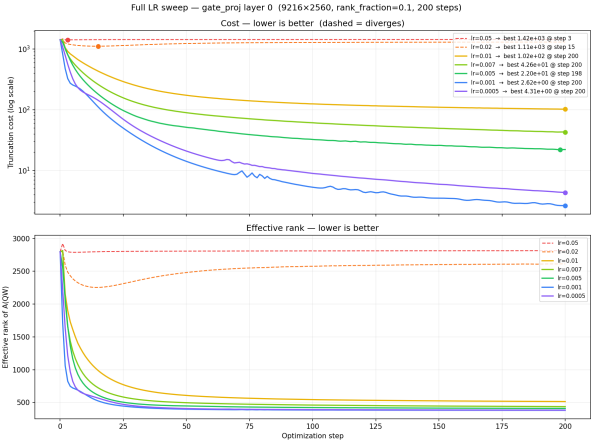
Based on ongoing work by our supervisor Tristan Till (sebis, TUM).

Progress
Tuning the disentanglement

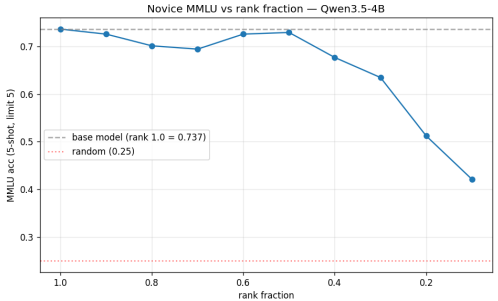
- Performance optimizations for scale.
- Learning-rate sweep for stability.

Takeaway

Large rates diverge; a mid-range rate minimizes both cost and effective rank.



Progress
Quality vs. compression



- MMLU accuracy holds near the base model down to rank fraction ≈ 0.5.
- Below that, quality drops sharply — the Novice becomes degenerate near rank 0.1.
- Gives us a controllable knob for how much weaker the Novice should be.

Outline

1 Introduction

2 Related Work

3 Progress

4 Outlook

5 References

Outlook
Next steps

- Run experiments with the **Qwen3.5-4B** model, generating rollouts on a **math reasoning dataset**.
- **Quantitative & qualitative evaluation** of which specific tokens drive divergence between the Master and the Novice.
- Explore how to **integrate this signal into a reinforcement learning setup**.



Outline

1 Introduction

2 Related Work

3 Progress

4 Outlook

5 References

References

- Longwen Wang et al. (2026). *Beyond Binary: Turning Partial Success into Dense Verifiable Rewards for Reinforcement Learning in Code Generation*. arXiv:2601.03525
- Yuanda Xu et al. (2026). *TIP: Token Importance in On-Policy Distillation*. arXiv:2604.14084
- Shenzhi Wang et al. (2025). *Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning*. arXiv:2506.01939
- Ganqu Cui et al. (2025). *Process Reinforcement through Implicit Rewards*. arXiv:2502.01456